Deep Learning-Based Crowd Density Estimation and People Counting Using Computer Vision

An End-to-End Computer Vision Framework for Automated Crowd Safety Monitoring

A Report

Submitted

in partial fulfillment of the requirements for the course

Advanced Artificial Neural Networks (AAN)

22ADC51N

By

B. PRANEETH REDDY (160124771028)

Under the guidance of

Dr. Shobarani Salvadi

Assistant Professor, AI&DS

Department of Artificial Intelligence and Data Science

Chaitanya Bharathi Institute of Technology (A)

Hyderabad - 500075

October 2026

Abstract

This project presents an offline computer-vision application for estimating crowd density and counting people in images and video. A pretrained Ultralytics YOLO26-nano detector (COCO class 0, person) localizes individuals; the system then computes a pixel-area-normalized density index, a mask-based surface occupancy percentage, a 3×3 spatial dispersion analysis, and a 2D Gaussian density heatmap. For video, a lightweight IoU-based tracker assigns persistent identifiers so that instantaneous counts can be separated from cumulative unique counts. Each scene is classified as LOW, MODERATE, HIGH, or CRITICAL using configurable thresholds on person count, density index, and occupancy. The pipeline is implemented in Python with PyTorch, OpenCV, and Streamlit, selects CUDA or CPU execution automatically, and exports annotated images, processed video, time-series plots, and JSON reports. No model was trained or fine-tuned in this project, and no quantitative accuracy benchmark was supplied with the project materials; this report therefore describes the design and the analytically derived behavior of the classification rule, and marks measured results as placeholders. The system is a decision-support aid and should not be used as the sole basis for safety-critical intervention.

1. Introduction

Mass gatherings at transit hubs, pilgrimage centres, airports, stadiums, and public squares can develop sudden density surges. Manual CCTV monitoring is difficult to sustain over long periods, and entry/exit counters report only totals, not how people are distributed inside a space. A raw headcount is also incomplete on its own: the same number of people can be harmless in an open square and hazardous in a narrow stairwell.

This project addresses the problem with an end-to-end offline pipeline. A pretrained YOLO26-nano network detects people in uploaded images or video. The system then derives a person count, a normalized density index, a surface occupancy percentage, and a spatial heatmap, tracks individuals across video frames, and maps the result to one of four operational risk tiers (LOW, MODERATE, HIGH, CRITICAL). Results are presented through a local Streamlit interface and exported as images, video, plots, and JSON files.

The emphasis of the work is on the analytical layer built on top of a pretrained detector: density metrics, occupancy, heatmap generation, tracking, and risk classification. The sections that follow describe the problem and objectives, the input handling, the architecture, the formulations used, the implementation, the evaluation evidence available, and the limitations of the approach.

1.1 Problem Statement

Develop a computer-vision system that detects and counts people in an image or video, quantifies how densely they are packed using normalized density metrics and a spatial heatmap, avoids double-counting individuals across video frames, and classifies the crowd condition into LOW, MODERATE, HIGH, or CRITICAL tiers, running locally on standard CPU or GPU hardware.

1.2 Objectives

- Detect people in a scene using a pretrained YOLO26 model restricted to the COCO person class.
- Count people: instantaneous counts per frame and cumulative unique counts through tracking.
- Estimate crowd density with a normalized density index and a mask-based surface occupancy.
- Generate 2D Gaussian spatial density heatmaps blended over the scene.
- Maintain persistent person identifiers across video frames.
- Classify crowd risk into LOW, MODERATE, HIGH, and CRITICAL tiers.
- Deliver image and video processing through an interactive Streamlit web application.

2. Dataset and Preprocessing

No dataset was collected and no model was trained for this project. Person detection relies on the pretrained Ultralytics YOLO26-nano weights (yolo26n.pt), whose checkpoint metadata references the COCO dataset configuration. Only the person class (COCO class ID 0) is retained at inference time.

Inputs are user-supplied images (.jpg, .jpeg, .png, .bmp, .webp) and videos (.mp4, .avi, .mov, .mkv, .webm). Each input is validated before analysis: an image must decode correctly and be at least 10×10 pixels, and a video must exist, be non-empty, and yield a decodable first frame. A 150 MB file-size warning threshold is configured.

Preprocessing is deliberately light. Images are analyzed at their native resolution. Video frames are down-scaled with area interpolation when the longer side exceeds 1280 pixels, and only every Nth frame is processed (default N = 2, adjustable from 1 to 8) to keep CPU throughput practical. The default detection confidence threshold is 0.35 and the default NMS IoU threshold is 0.45; the sidebar allows 0.10–0.90 and 0.20–0.80 respectively.

For demonstration, sample_data_generator.py prepares validation assets: a synthetic empty-street image, sparse and dense composite crowd images built from pedestrian crops extracted by the detector from OpenCV's public vtest.avi surveillance clip, and a 120-frame MP4 transcoded from that clip. Because these images are composites and no per-image person annotations were supplied, they serve as functional test inputs rather than ground-truth benchmarks.

3. System Architecture

3.1 Detection Backbone

YOLO26-nano (yolo26n.pt) is a single-stage convolutional detector that locates objects in one forward pass. The PersonDetector class (detector.py) loads the weights from the models/ folder when cached, otherwise requests them by name and copies them to models/ for later offline use. The device is selected automatically: CUDA when a GPU is available, otherwise CPU. Inference is restricted to the person class and uses the configured confidence and NMS thresholds; each detection yields a box, confidence, centre, and area.

3.2 Density Estimation and Risk Classification

The CrowdDensityEstimator class (density.py) converts boxes into a DensityReport with the person count N, density index ρ, persons per megapixel, occupancy Ω from a Boolean union mask (overlapping boxes are not counted twice), and a 3×3 grid analysis whose coefficient of variation labels the scene Uniform, Moderately Clustered, or Highly Clustered / Bottlenecked. A tier is assigned only when count, density index, and occupancy all fall within its limits; otherwise the next tier is tested, and anything beyond HIGH is CRITICAL. A scene with no detections is LOW.

3.3 Tracking and Heatmap Modules

The PersonTracker class (tracker.py) matches new detections to existing tracks greedily from the highest IoU downward, accepting a match only when IoU ≥ 0.30. Unmatched detections start new tracks and increment the cumulative unique count; tracks unseen for more than 15 frames are removed, and recent centre points are kept for trajectory trails. Association uses IoU only, with no motion model or appearance descriptor.

The visualization module (visualization.py) accumulates a 61×61 Gaussian kernel ($\sigma = 25$) at every person centre, normalizes and blurs the map, and applies a colormap (JET by default), blended at $\alpha = 0.50$ only where heat is non-negligible. It also draws boxes, confidence tags, track IDs and trails, a HUD badge, the 2×2 composite panel, and time-series plots.

3.4 End-to-End Workflow

1. Upload an image or video in the Streamlit interface and set the confidence, NMS, colormap, frame-skip, and tracking options.
2. Validate the file and decode it to BGR frames; down-scale video frames whose longer side exceeds 1280 pixels.
3. Run YOLO26 inference on the selected device and keep person detections above the confidence threshold after NMS.
4. In video mode, update the tracker to assign persistent IDs and maintain the cumulative unique count.
5. Compute N, ρ, Ω, the 3×3 spatial distribution, and the crowd risk tier.
6. Generate the Gaussian heatmap and blended overlay, and draw boxes, IDs, and the HUD.
7. Display the results in the application tabs and dashboard.
8. Export the annotated image, 2×2 composite, processed MP4 (re-encoded to H.264 when ffmpeg is available), time-series plot, and JSON summaries.

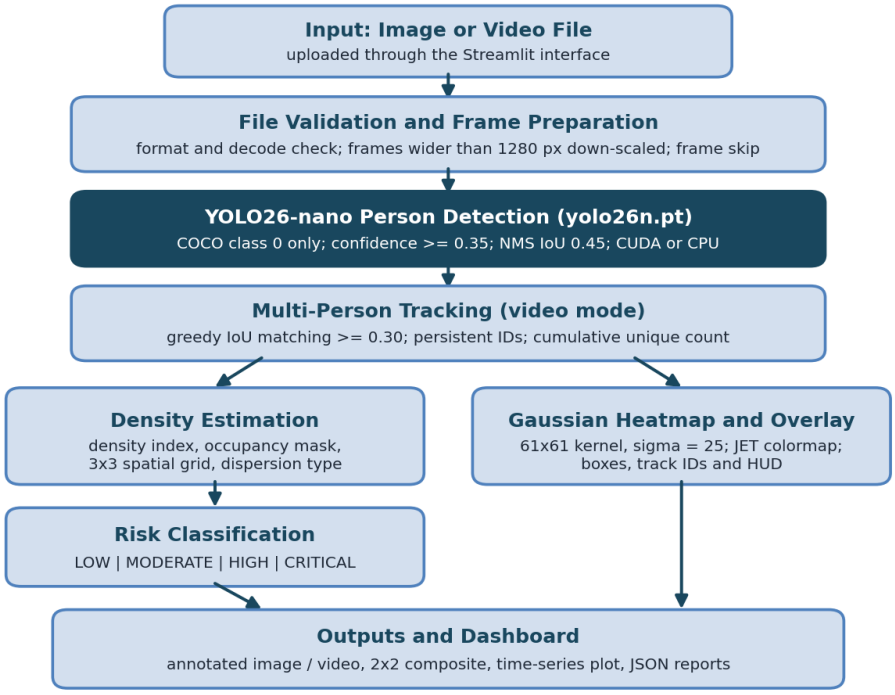


Figure 3.1: End-to-end processing pipeline of the system

4. Methodology and Mathematical Formulation

No training or fine-tuning was performed: the detector is used with its pretrained COCO weights, so the methodological content of the project is the analytical layer built on the detections. Its main formulations are given below.

$$\rho = N \times 10{,}000 / (W \times H)$$

The density index ρ is the number of detected persons N (confidence at or above the threshold) per 100×100 pixel window of a W×H frame. It is normalized by pixel area, not by physical ground area.

$$\Omega = (|M| / (W \times H)) \times 100\%$$

The occupancy Ω uses a Boolean mask M that is true wherever any bounding box covers a pixel, so overlapping boxes are counted once and Ω cannot exceed 100%.

$$D(x, y) = \Sigma \exp(-[(x - c_{x,i})^2 + (y - c_{y,i})^2] / 2\sigma^2)$$

The heatmap D sums an isotropic Gaussian (σ = 25 pixels, 61×61 kernel) centred at each person; the map is then divided by its own maximum, so colors express relative density within a frame.

$$CV = \sigma_{grid} / \mu_{grid}$$

The coefficient of variation of the 3×3 grid counts labels dispersion: below 0.45 is Uniform, below 0.90 is Moderately Clustered, and otherwise Highly Clustered / Bottlenecked. Track association uses IoU(A, B) = Area(A ∩ B) / Area(A ∪ B). Table 4.1 lists the default operating parameters.

Parameter	Default value
Detection confidence threshold	0.35
NMS IoU threshold	0.45
Tracker IoU threshold	0.30
Tracker max lost frames	15
Frame skip (video)	2 (adjustable 1–8)
Max processing dimension	1280 px
Heatmap kernel / σ / α	61 / 25 / 0.50

Table 4.1: Default operating parameters (config.py and app.py)

5. Implementation and Application

The implementation is organized into the following files:

- app.py — Streamlit interface: sidebar controls, Home, Image Analysis, Video Analysis, Dashboard, and Technical Docs pages, and download buttons.
- config.py — paths, model name, risk thresholds, heatmap, video, and tracker settings.
- detector.py — PersonDetector: model loading, device selection, and person-only inference.
- density.py — CrowdDensityEstimator: density index, occupancy mask, 3×3 distribution, and tier classification.
- tracker.py — PersonTracker: IoU association, persistent IDs, and cumulative unique count.
- video_processor.py — frame loop, frame skipping, rendering, time-series logging, H.264 export, and summary JSON.
- visualization.py — HUD, boxes, Gaussian heatmap, overlay, 2×2 composite, and metrics chart.
- utils.py — device detection, file validation, unique filenames, and JSON saving.
- sample_data_generator.py — sample validation images and video.
- requirements.txt — ultralytics, torch, torchvision, opencv-python, streamlit, numpy, matplotlib, pandas, pillow, scipy, imageio, imageio-ffmpeg.
- run_app.bat — Windows launcher.

The application is intended to run locally. On Windows, run_app.bat activates a local virtual environment if present, generates the sample assets when missing, and launches the command python -m streamlit run app.py, which serves the interface at http://localhost:8501. The weights file yolo26n.pt is expected in the models/ folder; if absent, the detector asks Ultralytics to retrieve it, which needs an internet connection once. Outputs are written to the outputs/ folder (annotated images, heatmaps, composites, analysis JSON, processed video, metrics plots, and video summary JSON).

6. Evaluation and Results

No benchmark metrics, annotated test sets, test-suite logs, or recorded run outputs were supplied with the project materials. This section therefore reports the configured classification rule, what it implies analytically, and placeholders for measurements that must be taken by running the application.

Risk tier	Max count	Max density index ρ	Max occupancy Ω
LOW	9	0.035	8.0%
MODERATE	24	0.085	18.0%
HIGH	49	0.160	32.0%
CRITICAL	above HIGH	above HIGH	above HIGH

Table 6.1: Risk tier limits configured in config.py (all three criteria must hold)

Because ρ is divided by pixel area, the density-index limit often becomes the binding rule. Table 6.2 gives the largest person count that still satisfies each ρ limit at common frame sizes (computed from the configured thresholds; the count and occupancy criteria can only lower these values).

Frame size	LOW (ρ ≤ 0.035)	MODERATE (ρ ≤ 0.085)	HIGH (ρ ≤ 0.160)
640 × 480	1	2	4
960 × 640	2	5	9
1280 × 720	3	7	14
1920 × 1080	7	17	33

Table 6.2: Maximum persons per tier implied by the density-index limit alone

Interpretation: at 960×640, ten or more detected people already exceed the HIGH density-index limit and are classified CRITICAL, even though the nominal count band for CRITICAL starts at 50. The person-count bands quoted in the project documentation are reached only in large frames, and no calibration of these limits against physical density (persons per square metre) or ground truth was supplied.

Measured results to be completed after running the application on the intended test inputs:

Item	Result
Person-count error versus manual count on test images	[PLACEHOLDER: to be measured]
Video processing speed (FPS) on CPU and GPU	[PLACEHOLDER: hardware and FPS]
Outcome of tests/test_pipeline.py (file not supplied)	[PLACEHOLDER: pass/fail log]
Screenshots of the interface and 2×2 composite output	[PLACEHOLDER: add figures]

Table 6.3: Measured results (not supplied; to be completed)

Output artifacts: for each image analysis the application writes *_annotated.jpg (boxes, confidence tags, HUD), *_heatmap.jpg, *_combined.jpg (2×2 composite), and *_analysis.json. For each video it writes processed_video_*.mp4, crowd_metrics_plot_*.png (count and density index versus time), and video_summary_*.json (frames processed, processing FPS, average and maximum count, unique people, and the percentage of frames in each risk tier).

7. Limitations and Failure Modes

- Pixel-based density: ρ and Ω are normalized by frame pixel area, not ground area, so the same scene captured at a different resolution or zoom yields different values; the tier limits are not calibrated to persons per square metre.
- Perspective distortion: people near the camera occupy more pixels than distant people, and no ground-plane homography is applied.
- Occlusion: in dense crowds bodies overlap and the detector undercounts; the occupancy mask measures box area, not visible body area.
- Tracker limits: IoU-only greedy matching without motion or appearance cues, aggravated by frame skipping, can switch or fragment IDs and inflate the cumulative unique count, which should be read as an estimate.
- Relative heatmap: the map is normalized by its own maximum, so a sparse scene can still show intense color.
- Heuristic thresholds: tier limits were set by design and have not been validated against incident data or expert-labelled scenes.
- Detector dependence: small, distant, or partially visible people and unusual poses may be missed; the confidence threshold trades missed detections against false ones.
- Test inputs: the supplied sample images are composites, not annotated benchmarks, and no accuracy figures were supplied.
- Licensing and privacy: the checkpoint metadata lists the AGPL-3.0 licence, which should be reviewed before deployment, and video of people raises privacy obligations.

8. Future Enhancements

- Evaluate the detector and the density rule on annotated crowd-counting datasets, reporting count error, per-scene results, and confidence intervals.
- Add ground-plane homography for top-down perspective correction and densities expressed in persons per square metre.
- Calibrate the tier thresholds against physical density and expert-labelled scenes instead of pixel-normalized values.
- Integrate hybrid density-regression models (for example CSRNet) for ultra-dense views where box detectors undercount.
- Strengthen tracking with a motion model and appearance features, and compare against trackers available in the Ultralytics library.
- Export the pipeline through TensorRT or ONNX for edge deployment on devices such as NVIDIA Jetson.
- Record and automate regression tests, with saved configurations, software versions, and result logs for reproducibility.
- Add alerting for HIGH and CRITICAL tiers and keep human review in the loop, together with privacy safeguards for stored footage.

9. Conclusion

The project delivers a complete offline pipeline for crowd density estimation and people counting. A pretrained YOLO26-nano detector provides person localization; the added analytical layer computes a pixel-area-normalized density index, a union-mask occupancy, a 3×3 dispersion analysis, a Gaussian heatmap, IoU-based tracking, and a four-tier risk classification, all exposed through a Streamlit application that adapts to CUDA or CPU hardware and exports images, video, plots, and JSON reports. The analysis in this report also shows that the density index depends on frame resolution and that the configured tier limits are heuristic, so tier labels should be read as indicative. Reliable deployment requires annotated evaluation, threshold calibration, perspective correction, and robustness testing. The system is best positioned as a monitoring aid that supports, rather than replaces, human judgment.

References and Source Materials

- Project presentation: Deep Learning-Based Crowd Density Estimation and People Counting Using Computer Vision (supplied PowerPoint and PDF).
- Project source code: app.py, config.py, detector.py, density.py, tracker.py, video_processor.py, visualization.py, utils.py, and sample_data_generator.py (supplied files).
- Project documentation: README.md and PROJECT_EXPLANATION.md (supplied files).
- Model weights: yolo26n.pt, Ultralytics YOLO26-nano detection checkpoint (supplied file).
- Libraries listed in requirements.txt: Ultralytics, PyTorch, OpenCV, Streamlit, NumPy, Matplotlib, Pandas, Pillow, SciPy, and imageio.
- Sample video: vtest.avi from the OpenCV repository, downloaded by sample_data_generator.py.

Note: README.md and PROJECT_EXPLANATION.md describe a YOLOv8 backbone in places; config.py and the supplied weights file specify YOLO26-nano, which this report follows.